

SPEC-DRIVEN *De cero a maestro.* DEVELOPMENT CON GAIA



Créditos

Título: *Spec-Driven Development con GAIA: De Cero a Maestro*

Autora: Cristina Cachero

Edición / versión: v1.0

Fecha: 29 de enero de 2026

Licencia: Creative Commons Atribución 4.0 Internacional (CC BY 4.0).

Se permite copiar, redistribuir, remezclar, transformar y crear a partir de este material para cualquier finalidad, incluso comercial, siempre que se reconozca adecuadamente la autoría, se incluya un enlace a la licencia y se indique si se han realizado cambios.

Licencia (enlace): <https://creativecommons.org/licenses/by/4.0/deed.es>

Atribución recomendada:

Cristina Cachero, *Spec-Driven Development con GAIA: De Cero a Maestro*, 2026. Licencia CC BY 4.0.

Solicitud para facilitar trazabilidad y transferencia:

Si vas a reutilizar este material (total o parcialmente), te agradeceré que lo hagas preferentemente mediante un *fork* del repositorio en GitHub y abriendo un *issue* con un enlace a tu uso/adaptación. Esto me ayudará a documentar reutilizaciones y evidencias de transferencia.

Índice general

Índice de figuras	7
Capítulo 1. Gobernanza de artefactos	9
1.1. Scope: Global vs Workspace	9
1.1.1. Qué significa “Global” vs “Workspace”	9
1.1.2. Dónde se ve en la UI	9
1.1.3. Dónde se guardan físicamente los artefactos	9
1.1.4. Regla práctica para tu proyecto SDD	9
1.2. Creación de skill, regla y workflow para la gobernanza de los artefactos SDD	10
1.2.1. Skills (Habilidades)	10
1.2.2. Rules (Reglas)	15
1.2.3. Workflows (comandos)	16
1.3. Gobernanza de artefactos SDD en la práctica	19
1.4. Ejemplo práctico	20
1.4.1. Averiguar los artefactos necesarios	20
1.4.2. Crear los artefactos	21
1.4.3. Ejecutar el proceso	21
Capítulo 2. G.A.I.A.	23
2.1. Repositorio base (para fork y clonar)	23
2.2. Mapa de Flujos Operativos básicos	23
2.3. Intención: implementación de una feature	23
2.3.1. Fase 1: Definición del Producto (Specs)	24
2.3.2. Fase 2: Planificación	24
2.3.3. Fase 3: Implementación	25
2.3.4. Fase 4: Cierre	26
2.4. Intención: arreglar errores (Bugs)	27
2.4.1. Fase 1: Triage Mental (Zero-Touch)	27
2.4.2. Fase 2: Preparación de Entorno (Git Safe-Zone)	27
2.4.3. Fase 3: Formalización de Artefactos (SDD)	27
2.4.4. Fase 4: Delegación de Ejecución	28
2.5. Intención: refactorización de la interfaz de Usuario (UI)	28
2.5.1. Refactorización según identidad de marca:	28
2.6. Intención: cambios en artefactos SDD (PRD, Arquitectura, Reglas)	29
2.7. Intención: technical discovery	30
2.8. Intención: auditoría de seguridad	30
2.9. Secuencia SDD “Pro” al iniciar algo nuevo	31
2.10. Conclusión: ecosistema SDD Pro “blindado”	31
2.11. Resumen final de GAIA: mapa completo de flujos y artefactos SDD propuestos	31
2.11.1. Constitución (prerrequisito permanente)	31
2.11.2. Flujo base para funcionalidad nueva (SDD “end-to-end”)	32

2.11.3. Flujos operativos por tipo de cambio (gobernanza diaria)	32
Capítulo 3. GAIA-TDD	35
3.1. Idea fundamental: La Prueba es la Evidencia	35
3.2. El ciclo de vida: Double Loop TDD	35
3.2.1. Bucle externo (Outer Loop): Validación de usuario.....	35
3.2.2. Bucle interno (Inner Loop): Lógica y corrección.....	36
3.3. Implementación operativa de las pruebas en GAIA.....	36
3.3.1. Fase A: Diseño y estrategia (Planning)	36
3.3.2. Fase B: Ejecución Red/Green/Refactor (Inner Loop).....	36
3.3.3. Fase C: Garantía de calidad E2E (Outer Loop).....	37
3.3.4. Fase D: Gestión de errores (Reproduction First)	37
3.4. La pirámide de testing en GAIA.....	37
3.5. Glosario de mejores prácticas adoptadas.....	38
3.6. Ejemplo guiado: regla de negocio “no se puede crear un evento sin fecha”	38
3.6.1. Paso 1: RED (test que falla).....	38
3.6.2. Paso 2: GREEN (implementación mínima)	38
3.6.3. Paso 3: REFACTOR (mejor diseño, mismo comportamiento)	39
3.7. Actividad práctica	39
3.8. Cierre: por qué GAIA-TDD es más que TDD	41

3.1. Double Loop TDD en GAIA: bucle externo (E2E) y bucle interno (unit/integration).....	35
3.2. Pirámide de testing en GAIA: coste y velocidad determinan qué se prueba y dónde....	37

Capítulo 1 Gobernanza de artefactos

1.1. Scope: Global vs Workspace

En Antigravity tienes dos scopes: **Global** y **Workspace**. En este contexto, **scope** (ámbito) significa: **dónde se guarda el artefacto y, por tanto, en qué proyectos se aplica.**

1.1.1. Qué significa “Global” vs “Workspace”

- ❖ **Global:** se aplica a todos tus proyectos/workspaces en ese ordenador (tus preferencias personales). Se guarda en tu carpeta de usuario.
- ❖ **Workspace:** se aplica solo al proyecto actual (la carpeta que tienes abierta como workspace / raíz del repositorio). Se guarda dentro del propio proyecto (ideal para compartirlo por Git con el equipo).

1.1.2. Dónde se ve en la UI

Para ver o elegir el scope desde la interfaz:

1. En el panel del agente, arriba a la derecha, pulsa . . .
2. Elige Customizations.
3. Verás Rules y Workflows.
4. Dentro, al crear, verás botones +Workspace y +Global.

1.1.3. Dónde se guardan físicamente los artefactos

Antigravity guarda los artefactos en las siguientes rutas (tú puedes crearlas y editarlas manualmente):

- ❖ **Global rule:** `~/ .gemini/GEMINI.md`
- ❖ **Workspace rules:** `<tu-proyecto>/ .agent/rules/`
- ❖ **Global workflows:** `~/ .gemini/antigravity/global_workflows/ . . .`
- ❖ **Workspace workflows:** `<tu-proyecto>/ .agent/workflows/`
- ❖ **Global skills:** `~/ .gemini/antigravity/skills/ . . .`
- ❖ **Workspace skills:** `<tu-proyecto>/ .agent/skills/`

Atención

En el texto literal de rutas, recuerda que el carácter `~` representa tu carpeta de usuario.

1.1.4. Regla práctica para tu proyecto SDD

En tu proyecto SDD, lo recomendable casi siempre es:

- ❖ **Workspace:** todo lo que sea gobernanza del proyecto / SDD / convenciones del repositorio (porque quieres versionarlo y que se aplique igual a cualquiera que clone el repo).

❖ **Global:** solo tus preferencias personales transversales (estilo de comunicación, pequeños “defaults”) que no quieres duplicar en todos los repos.

1.2. Creación de skill, regla y workflow para la gobernanza de los artefactos SDD

Para que veáis la relación entre skills, reglas y workflows, aquí tenéis un ejemplo de cada uno y cómo se interrelacionan para asegurar la gobernanza de los artefactos que guían el entorno de desarrollo SDD (Rules, Workflows, Skills).

1.2.1. Skills (Habilidades)

Un skill es una “biblioteca de referencia”: contiene el conocimiento estático y detallado que permite extender las capacidades del agente con conocimiento, convenciones y recursos/scripts de manera **bajo demanda** (no siempre cargados). Un script es útil cuando la tarea requiere **procedimientos complejos**, plantillas, ejemplos, scripts auxiliares o “know-how” que es mejor encapsular. Lo interesante de las skills es que el agente puede seleccionar automáticamente el skill en función de la intención del usuario en base a su intención (a través de su description).

Como ejemplo, en 1.1 tenéis una skill que proporciona el conocimiento estático y detallado que, ante la necesidad de añadir nuevos artefactos de SDD al proyecto, permite al agente de Antigravity decidir con criterio entre crear una Regla, un Workflow o un Skill, y los criterios de calidad para cada uno. Si queréis incluirla en vuestro proyecto, esta especificación Markdown se debe guardar como `.agent/skills/antigravity-artifact-governance/SKILL.md`.

```

1 # Antigravity - Decision & Quality Guide: Rules vs Workflows vs Skills
2
3 ## Objective
4 This document defines when content should go into a Rule (and its scope),
5 a Workflow, or a Skill in Antigravity, and establishes quality
6 requirements + checklists to validate each artifact.
7
8 ---
9
10 ## Quick decision (30 seconds)
11
12 ### Use a Rule when...
13 - It's a permanent constraint/criterion ("MUST / MUST NOT") that should
14 influence any relevant generation or modification.
15 - It's reusable context at prompt level: style, architecture, security,
16 conventions, limits.
17 - It's not a sequence; it's a norm.
18
19 Typical examples
20 - "Don't work directly on 'main'; create feature/fix branches."
21 - "Frontend: use semantic tokens, avoid hex values."
22 - "Backend: use repositories; direct DB access from controllers is forbidden."

```

Use a **Workflow** when...

- It's a **repeatable process** with **steps** ("first... then... finally...") that you want to execute **on demand**.
- It should guide the **trajectory** of work (plan -> actions -> validation -> close).
- It's useful as a command-invokable "macro".

Typical examples of workflows

- "/open-pr" (sync with main -> green CI -> push -> open PR).
- "/respond-pr-comments" (read comments -> group -> apply changes -> rerun tests).

Use a **Skill** when...

- You want to extend capabilities with **knowledge + conventions + resources/scripts** in an **on-demand** way (not always loaded).
- The task requires **complex procedures**, templates, examples, helper scripts, or "know-how" that's best encapsulated.
- You want the agent to automatically select the skill based on user intent (via its 'description').

Typical examples

- "Generate unit tests following your conventions (and run them)."
- "Scaffold endpoints + validations + contracts."
- "Review PR with checklist + lint/format/test scripts."

1) RULES

What they are

Persistent constraints and guidelines (Markdown) that condition the agent's behavior.

Scope

- **Global Rule**: applies to all your workspaces.
- **Workspace Rule**: applies only to the current repo/workspace.

When to choose Global vs Workspace

- **Global** if the rule reflects *your general way of working* (e.g., commit style, communication preferences, personal security policies).
- **Workspace** if it's *project-specific* (tech stack, architecture, naming, routing, internal conventions).

Activation modes (use the minimum needed)

- **Manual**: activated only when explicitly mentioned.
- **Always On**: always active (use only for truly universal rules).
- **Model Decision**: provide a description and the model decides if it applies (

useful when context-dependent).

- ****Glob****: applied via file patterns (useful for tech/layer-specific rules: 'src/**/*.ts', 'backend/**', etc.).

What should NOT be in a Rule (anti-patterns)

- Long "step-by-step" processes (use Workflow instead).
- Procedures with many cases and resources (use Skill instead).
- Redundant text or huge templates (use '@mentions' to reference other files).

Quality characteristics for Rules

- ****Concise and testable****: each bullet should be verifiable.
- ****Normative****: use clear formulations ("MUST", "MUST NOT", "SHOULD").
- ****Unambiguous****: if context-sensitive, define condition or move to Workflow/Skill.
- ****Modular****: better to split into small rules than a giant "mega-rule".
- ****Referential****: use '@...' to point to source docs when needed (avoid duplication).

2) WORKFLOWS

What they are

Processes (Markdown) with a ****sequence of steps****, invokable via '/workflow-name'.
Can call other workflows.

Scope

- ****Global Workflow****: reusable in all your workspaces.
- ****Workspace Workflow****: tied to a repo/project.

When to use Workflow (and not Rule)

- When the value lies in the ****execution order**** (checkpoints, validations, gates).
- When you want a "one-liner" command to launch a repeatable protocol.

What should NOT be in a Workflow (anti-patterns)

- Permanent restrictions (use Rule instead).
- Heavy "domain knowledge" (use Skill instead).
- Steps that depend on unavailable data without specifying how to get it.

Quality characteristics for Workflows

- ****Step = verifiable action**** (ideally with expected output/artifact).
- ****Explicit checkpoints****: when to review, when to stop on error.
- ****Mental idempotence****: re-running shouldn't break things (or should explain how to resume).
- ****Ambiguity handling****: *if info is missing for a step*, the workflow should say

"ask X before continuing".

3) SKILLS

What they are

Packages (folders) with 'SKILL.md' + optional resources (scripts, examples, templates). Loaded **only when relevant** based on their 'description'.

Scope

- **Global Skill**: available in all projects.
- **Workspace Skill**: specific to a repository.

When to use Skill (and not Workflow)

- When the content is **reusable knowledge/procedure** that applies **across multiple tasks** with variations.
- When you have **assets** worth grouping: 'scripts/', 'examples/', 'resources/'.
- When you want an "ability" the agent can choose based on user intent.

Recommended minimal structure

- 'SKILL.md' with:
 - **YAML Frontmatter** ('name', 'description')
 - **Markdown body** with: goal, when to use, procedure, examples, constraints, verification
- Optional folders: 'scripts/', 'examples/', 'resources/'

Quality characteristics for Skills

- **Focused**: one skill = "does one thing well".
- **Reliable activation**: 'description' must have clear keywords and criteria (avoid vagueness).
- **Operational instructions**: concrete steps + expected outputs.
- **Scripts as black boxes**: explain how to run them and suggest '--help' if needed.
- **Decision tree** when multiple strategies exist.
- **Security**: explicit restrictions (e.g., no destructive actions, no exposed secrets).

General quality rules (apply to all)

1. **Zero operational ambiguity**

- If something can be interpreted in two ways, **require a question** (what data is missing, what preference to use).

2. **Concise but dense**

- Less text, more precision. Avoid filler.
- 3. ****Verifiable****
 - Every requirement should be checkable (via tests, CI, diff review, logs).
- 4. ****No duplication****
 - If it exists elsewhere, reference (or delegate), don't copy.
- 5. ****Separation of responsibilities****
 - Rule = norm; Workflow = sequence; Skill = capability + resources.

CHECKLISTS

A) Validation checklist - RULE

- [] The rule expresses ****constraints**** (not step-by-step processes).
- [] Each point uses normative verbs: ****MUST / MUST NOT / SHOULD****.
- [] It is ****unambiguous****: includes conditions ("if X then Y") or requires questions.
- [] It is ****testable**** (can be verified in the repo/PR).
- [] It's in the ****correct scope**** (global vs workspace).
- [] Has ****proper granularity**** (not a mega-rule).
- [] Doesn't repeat long texts: uses '@mentions' when appropriate.
- [] Doesn't include volatile data (tokens, credentials, temporary URLs).
- [] Meets size limits (if applicable) and is bullet-readable.

B) Validation checklist - WORKFLOW

- [] Has ****title, description****, and a list of ****numbered or clearly ordered steps****.
- [] Each step is a ****concrete action**** with an expected result.
- [] Defines ****checkpoints**** (review, stop conditions, rollback/resume).
- [] Handles ****missing information****: states what to ask before continuing.
- [] Avoids encoding "permanent norms" (those go in Rules).
- [] Reuses other workflows/skills when appropriate (no duplication).
- [] Includes basic verification (tests, CI, smoke checks, etc.).
- [] Is safe: no destructive actions without an explicit gate.

C) Validation checklist - SKILL

- [] Folder exists with 'SKILL.md' (mandatory).
- [] Correct frontmatter:
 - [] 'description' is ****specific**** and includes activation keywords.
 - [] 'name' (if used) is unique and consistent.
- [] Body includes:
 - [] ****When to use**** / when NOT to use.
 - [] ****Step-by-step procedure****.
 - [] ****Input/output examples**** (at least one).
 - [] ****Constraints**** ("do not") and security limits.
 - [] ****Verification**** (how to confirm it worked).

```

179 - [ ] If there's a 'scripts/' folder:
180 - [ ] Explains how to run them, suggests '--help'.
181 - [ ] Clearly defines required inputs and expected outputs.
182 - [ ] The skill is not "do-everything": **focused on a specific goal**.
183 - [ ] It's in the **correct scope** (global vs workspace).
184 - [ ] No secrets or sensitive data are exposed.
185
186 ---
187
188 ## Final heuristic (if unsure)
189 - If you want it **always on** -> Rule.
190 - If you want it **sometimes**, as a repeatable process -> Workflow.
191 - If you want it **sometimes**, and it's a "specialized capability" with resources
    -> Skill.

```

Listing 1.1. SKILL.md para gobernar artefactos SDD

1.2.2. Rules (Reglas)

Las reglas actúan como la “Constitución” del proyecto: definen restricciones/criterios permanentes (MUST/MUST NOT) que deberían influir cualquier generación o modificación relevante del proyecto. Constituyen un contexto reutilizable a nivel de prompt. Ejemplos típicos de reglas son ficheros sobre arquitectura, seguridad, convenciones de codificación, límites del proyecto, stack tecnológico, ... Es importante tener claro que las reglas no son secuencias, son normas. Siguiendo con nuestro ejemplo de definición de artefactos de gobernanza SDD, se puede definir una regla que indique que “siempre que tu tarea involucre modificar/crear una Regla, Workflow o Skill, debes consultar el procedimiento oficial recogido en la skill xxx”. Esta regla, cuyo cuerpo puede verse en 1.2 se debe guardar como `.agent/rules/artifact-governance.md`.

```

1 # Rule - Artifact governance (Rules / Workflows / Skills)
2
3 When the task involves creating or updating any **Rule**, **Workflow**, or **Skill**:
4
5 - **MUST** open and follow: @/.agent/skills/antigravity-artifact-governance/SKILL.md (decision criteria + checklist) before writing anything.
6 - **MUST** ask targeted questions and stop if there is any ambiguity about: (a) whether it's Rule/Workflow/Skill, (b) scope (global vs workspace), or (c) activation mode / invocation.
7 - **MUST** keep artefacts concise and modular; split oversized docs; avoid duplication by referencing shared content via '@mentions'.
8 - **MUST** choose the **least-powerful activation** for Rules: Manual by default; Model Decision if conditional; Glob if file-scoped; Always On only if truly universal.
9 - **MUST** include an explicit verification step/section and ensure the relevant checklist is satisfied before considering the artefact "done".

```

```

- **MUST** keep each file within platform limits (<= 12,000 characters). If
  exceeded, split and reference.

```

Listing 1.2. Ejemplo de regla para crear artefactos SDD

1.2.3. Workflows (comandos)

Por último, un workflow en AntigraVity es lo mismo que un comando en otros IDEs como Cursor.

21 - It's a ****repeatable process**** with ****steps**** ("first... then... finally...") that you want to execute ****on demand****. 22 - It should guide the ****trajectory**** of work (plan -> actions -> validation -> close). 1.2.1.1.

12 Universidad de Alicante Desarrollo de software con IA 2025 59 CAPÍTULO 1. GOBERNANZA DE ARTEFACTOS1.2. CREACIÓN DE SKILL, REGLA Y WORKFLOW PARA LA GOBERN 23 - It's useful as a command-invokable "macro".

Los workflows definen "protocolos de ejecución", es decir, procesos repetibles con pasos bien establecidos que quieres que el desarrollador pueda ejecutar bajo demanda. Debe dirigir la trayectoria del trabajo (planificación -> acciones -> validación -> cierre). Además, es invocable desde el agente (puede verse como una especie de macro). Usar workflows ayuda a garantizar que el agente no se salta ninguna fase del proceso.

Siguiendo con nuestro ejemplo, en 1.3 tenéis un workflow que implementa el proceso de gobernanza de artefactos SDD. Este workflow se debe guardar como `.agent/workflows/governance-check.md` y se invocará como `/governance-check <tarea que quieres que realice>`.

```

1 # Workflow: /governance-check
2
3 **Purpose:** Validate whether content belongs in a **Rule**, **Workflow**, or **Skill** (and the correct scope), and run the corresponding checklist before
  merging changes.
4
5 ---
6
7 ## What this workflow checks
8
9 1. **Correct artifact type**
10   - Rule = constraints / reusable context
11   - Workflow = repeatable sequence of steps invoked on demand
12   - Skill = reusable capability package (instructions + optional resources/
    scripts)
13
14 2. **Correct scope**
15   - Global vs Workspace
16
17 3. **Correct activation / invocation**
18   - Rules: Manual / Always On / Model Decision / Glob
19   - Workflows: invoked via '/name'
20   - Skills: discovery via 'description' + progressive disclosure

```

4. ****Quality gates****

- Concision, modularity, no duplication (use '@mentions')
- Ambiguity handling: ask questions before proceeding
- Verification included
- File size within limits

Required reference

- Open and follow: @/.agent/skills/antigravity-artifact-governance/SKILL.md

> If this file does not exist, STOP and request it.

Inputs (ask if missing)

- Which artifact(s) are being created/changed? (paths or PR diff)
- Intended outcome (1-2 sentences): what problem does this artifact solve?
- Scope: 'global' or 'workspace'
- If Rule: activation mode (Manual / Always On / Model Decision / Glob) and any glob patterns
- If Workflow: intended command name (e.g., '/governance-check') and when it should be used
- If Skill: folder name and the exact 'description' (keywords + "use when..." criteria)

Steps

Step 1 - Collect artifacts

1. List the files/changes to review (or request document content).
2. Identify any referenced files via '@mentions'.

****STOP and ask**** if you cannot locate the artifact content.

Step 2 - Classify each artifact (Rule / Workflow / Skill)

For each artifact, produce:

- Proposed type (Rule/Workflow/Skill)
- Justification in 2-4 bullets referencing the decision criteria in the governance skill

****STOP and ask**** if classification is ambiguous.

```

63
64 ---
65
66 ### Step 3 - Validate scope (Global vs Workspace)
67 For each artifact:
68 - Confirm whether it belongs to 'global' or 'workspace'
69 - If mismatch, propose the correct location and why
70
71 ---
72
73 ### Step 4 - Validate activation / invocation
74 ##### If Rule
75 - Confirm chosen activation mode is the **least-powerful** option that works
76 - If Glob is used, validate the pattern is specific and correct
77
78 ##### If Workflow
79 - Confirm the command name matches the intended usage (short, action-oriented)
80 - Confirm each step is an action with an expected outcome
81
82 ##### If Skill
83 - Confirm 'description' is specific, third-person, and contains reliable keywords
84 - Confirm "When to use / When not to use" is explicit
85
86 ---
87
88 ### Step 5 - Run the relevant checklist(s)
89 - Use the checklists from: @/.agent/skills/antigravity-artifact-governance/SKILL.
    md
90
91 Output results as:
92 - Pass / Needs changes / Fail
93 - A short "what to change" list
94
95 ---
96
97 ### Step 6 - Validate concision and structure
98 Check:
99 - No mega-files; split and reference if needed
100 - No duplicated policies across artifacts (prefer '@mentions')
101 - Clear MUST/MUST NOT language for Rules
102 - Steps are numbered and unambiguous for Workflows
103 - Skills are focused on one capability
104
105 ---
106
107 ### Step 7 - Verify "ambiguity protocol"

```

```

108 Confirm the artifact(s) explicitly instruct:
109 - Ask targeted questions when inputs are missing or unclear
110 - Stop before making irreversible or assumption-heavy changes
111
112 If missing, propose exact text to add.
113
114 ---
115
116 ### Step 8 - Produce final governance report
117 Return a report in this format:
118
119 ##### Summary
120 - Artifacts checked: ...
121 - Overall status: Pass / Needs changes / Fail
122
123 ##### Findings (per artifact)
124 - Type: ...
125 - Scope: ...
126 - Activation/Invocation: ...
127 - Checklist result: Pass / Needs changes / Fail
128 - Required changes (bullets):
129   - ...
130 - Optional improvements (bullets):
131   - ...
132
133 ##### Next actions
134 - If Pass: ready to merge
135 - If Needs changes / Fail: list the minimal patch plan (ordered)
136
137 ---
138
139 ## Stop conditions
140 - Missing artifact content
141 - Ambiguous intent, scope, or activation choice
142 - Conflicting constraints across artifacts
143 - Any request that would exceed file size limits without splitting

```

Listing 1.3. Workflow: /governance-check

1.3. Gobernanza de artefactos SDD en la práctica

He usado este ejemplo de gobernanza de artefactos SDD para ilustrar la diferencia entre reglas, skills y workflows debido a su utilidad para asegurar la calidad y coherencia de los artefactos que guían el entorno de desarrollo SDD.

Algunos ejemplos de situaciones en las que conviene invocar el workflow de gobernanza /governance-check (donde yo lo he invocado con éxito) son:

- ❖ `/governance-check.md @agent/rules` comprueba que todos mis ficheros de reglas estan correctamente especificados y que no hay contradicciones entre ellos.
- ❖ `/governance-check.md` Genera un archivo `git-conventions.md` para guiar el uso de git y github en mi proyecto. Dime ademas si debe ir como regla, como skill o como workflow. Quiero que el flujo de git y github se ejecute de manera transparente.
- /
- ❖ `governance-check.md` crees que es conveniente mover `UserStories.md` tambien al directorio `/specs?` Y `PRD.md`? O mejor dejarlos en reglas?
- ❖ `/governance-check.md` si tengo una skill especifica para crear skills, donde lo indico? en la rule `artifact-governance.md`, en la skill `antigravity-artifact-governance` o en el workflow `governance-check.md`?
- ❖ `/governance-check.md @.agent/mworkflows` revisa los workflows. estan bien definidos, sin ambigüedades, sin redundancias y sin inconsistencias?
- ❖ `/governance-check.md` quiero indicar que tanto specs como artefactos SDD como codigo y comentarios se generen en ingles. Como lo hago?
- ❖ `/governance-check.md` quiero generar tests E2E en playwright. Que artefactos SDD tengo que crear para que quien esta generacion?
- ❖ `/governance-check.md` al modificar una skill, el nombre del skill dentro de `skill.md` ha dejado de coincidir con el nombre de la carpeta. Arregla el error y dime que hay que hacer para asegurarnos de que no vuelva a ocurrir.
- ❖ `/governance-check.md` dime cual seria la secuencia de llamadas a workflows ideal para implementar un nuevo feature, y si me faltaria por definir alguna skill, regla o workflow para mejorar el flujo.

1.4. Ejemplo práctico

A continuación os muestro cómo he añadido a los artefactos SDD de mi proyecto una estructura completa regla-skill-workflow para la automatización de la adición de tests E2E en Playwright.

1.4.1. Averiguar los artefactos necesarios

El primer paso para automatizar este aspecto ha sido ejecutar el siguiente comando: `/governance-check` quiero generar tests E2E en playwright. ¿Qué artefactos SDD tengo que crear para que quién esta generacion?

La respuesta del agente ha sido: aplicando el modelo de gobernanza a tu necesidad de “Tests E2E con Playwright”, tienes que actualizar las reglas, crear un skill y crear un workflow.

Regla (Constraint/Norma):

Necesitas definir qué herramientas y prácticas son obligatorias.

Artefacto: `.agent/rules/techstack-frontent.md` (o `testing.md`).

Contenido: “MUST use Playwright for E2E”, “MUST use Page Object Model”, “MUST tag tests with @e2e”.

Skill (Capacidad Especializada):

Necesitas enseñar al agente cómo escribir buenos tests en Playwright eficientemente (con scripts, snippets, buenas prácticas de selectores).

Artefacto: `.agent/skills/playwright-e2e/SKILL.md`.

Contenido: guía de “Como crear un Page Object”, “Como manejar autenticacion”, scripts de `npm run`

test:e2e.

Workflow (Proceso Repetible):

Necesitas un comando para generar un test a partir de un ticket/historia.

Artefacto: `.agent/workflows/generate-e2e-test.md`.

Contenido: "1. Lee el ticket/historia en specs/features/. 2. Lee el Skill de Playwright. 3. Genera el fichero `.spec.ts` correspondiente".

1.4.2. Crear los artefactos

El siguiente paso es crear cada uno de estos artefactos siguiendo las guías y checklists del skill de gobernanza de artefactos SDD. Para ello, puedes volver a usar el comando `/governance-check.md` para cada uno de estos artefactos (regla, skill, workflow), pidiéndole al agente que te los genere.

Antes de finalizar, como realizar tests E2E es un proceso que os interesa tener optimizado en vuestros proyectos, os recomiendo que no os quedéis con la primera versión que os dé AntigraVity, sino que intentéis optimizar la skill.

Existen muchas formas de optimizarla: podéis pasársela a alguna persona experta en Playwright para que mejore la especificación, o podéis pedirle a la IA que la mejore por vosotros, idealmente usando un LLM distinto y tras haberle proporcionado información adicional y contexto relevante. En mi caso he seguido este segundo camino y he creado un notebookLM con información detallada sobre Playwright, incluyendo ejemplos de tests, patrones de diseño y anti-patrones. Esta información la he usado para pedirle a NotebookLM que criticara y mejorara el skill inicial de Playwright que había creado el agente (<https://notebooklm.google.com/notebook/d153b60e-ea22-4f24-8f04-c49d8060721c>). El resultado ha sido una serie de recomendaciones que he copiado y pegado en AntigraVity tras el comando `/governance-check`. A partir de esa info, el agente ha modificado el skill en base a dichas recomendaciones.

** Atención**

Recordad, la mejora de skills, rules y workflows es un proceso iterativo que se puede repetir tantas veces como queramos para ir puliendo los artefactos SDD que guían nuestro entorno de desarrollo. Como se trata de artefactos reutilizables entre proyectos, sin duda merece la pena dedicar tiempo a su mejora continua.

1.4.3. Ejecutar el proceso

El último paso es ejecutar el workflow que hemos creado para generar tests E2E en Playwright. Para ello, simplemente tenemos que invocar el comando `/generate-e2e-test <feature>` y el agente se encargará de leer la feature description y las historias de usuario, consultar la skill de Playwright y generar el fichero de test correspondiente siguiendo las reglas definidas en el proyecto.

Capítulo 2 G.A.I.A.

En el desarrollo de software, los procesos juegan un papel fundamental para garantizar la calidad, la eficiencia y la colaboración en los equipos. Un proceso bien definido no solo establece un marco claro para la ejecución de tareas, sino que también fomenta la alineación entre los miembros del equipo, reduce la ambigüedad y asegura que los objetivos del proyecto se cumplan de manera consistente. ¡En este capítulo, exploraremos juntos los diferentes procesos que se han definido dentro de la aproximación **G.A.I.A. — Governed AI for Interactive Applications** —.

Desde la planificación inicial hasta la entrega final, cada proceso está diseñado para optimizar el flujo de trabajo, mejorar la trazabilidad y garantizar que los artefactos generados cumplan con los estándares de calidad establecidos. Además, se presentarán ejemplos prácticos y recomendaciones para adaptar estos procesos a las necesidades específicas de tu equipo y proyecto.

A través de esta guía, aprenderás cómo estructurar y gestionar proceso de desarrollo software SDD de manera efectiva con GAIA, aprovechando al máximo las herramientas y metodologías más actuales en un entorno de desarrollo moderno y colaborativo.

2.1. Repositorio base (para fork y clonar)

URL del repositorio de GitHub para hacer fork y clonar: url del repositorio de github para hacer fork y clonar

Nota: contiene solo los artefactos SDD para usar con Antigravity para hacer un proyecto con API backend (framework FastAPI de python) y frontend REACT.

2.2. Mapa de Flujos Operativos básicos

En un proyecto SDD con IA usando GAIA, no todos los cambios son iguales. Un ajuste visual puede romper accesibilidad, un “fix rápido” puede introducir deuda técnica, y un cambio global puede desalinear el “estado mental” del proyecto (especificaciones vs. código). Para mantener **integridad arquitectónica** y **trazabilidad**, el proceso se gobierna mediante flujos operativos distintos según el **impacto** del cambio.

Definición 2.1

Un **flujo operativo** es un protocolo de trabajo (workflow + reglas y skills que lo apoyan) optimizado para un tipo de intención concreta. Su objetivo es minimizar improvisación, forzar validación (tests + revisión) y preservar trazabilidad (intención → ticket → commit → PR).

2.3. Intención: implementación de una feature

En GAIA, cuando la intención es implementar una nueva funcionalidad (feature), se sigue el siguiente ciclo operativo estable: **Especificación** → **Planificación** → **Ejecución Validada** → **Cierre del feature**. La idea central es que **no existe implementación sin especificación revisada** y **no existe código sin plan aprobado y verificaciones hechas**.

La definición de cada fase es la siguiente:

- ✿ **Especificación:** convertir una idea en documentos inequívocos.
- ✿ **Planificación:** transformar requisitos en tickets y planes atómicos por ticket.
- ✿ **Ejecución Validada:** implementar estrictamente el plan y validar con tests (TDD) y revisiones.
- ✿ **Cierre:** formaliza el cierre de una feature verificando que todos sus tickets estén completados, sus tests pasen, y sus objetivos de negocio se hayan cumplido según los specs.

2.3.1. Fase 1: Definición del Producto (Specs)

En esta fase convertimos una idea inicial en especificaciones que el sistema puede seguir **sin ambigüedad**.

El primer paso es, si la tenemos, incluir una visión general del producto que pretendemos construir en `specs/PRD.md` (Product Requirements Document). Este documento es el **contrato de más alto nivel** que existe entre la persona desarrolladora y el agente.

A continuación, invocaremos los siguientes workflows:

✿ **/plan-feature-descr-from-user-conversation:**

- ✿ La persona desarrolladora describe la idea (p. ej. “Quiero gestionar las noticias de la comunidad. [Aquí añadirá los detalles que conozca sobre cómo se debe realizar dicha gestión]”).
- ✿ El workflow genera `specs/features/[feature-slug]/feature-descr.md` (un “mini PRD”) con problema, actores y reglas de negocio.
- ✿ Además, el workflow actualiza el PRD.md global `specs/PRD.md`.
- ✿ **Control (Review Protocol):** *Validation Gate* explícito: el agente debe pedir que la persona desarrolladora valide el PRD y `feature-descr.md` antes de continuar, siguiendo la filosofía de human in the loop.

✿ **/plan-user-stories-from-features:**

- ✿ La persona desarrolladora añade la **mención** al documento de feature generado en el paso anterior (@).
- ✿ El workflow desglosa `feature-descr.md` en historias de usuario con escenarios Gherkin (Given/When/Then) y los guarda en `specs/features/[feature-slug]/user-stories.md`.
- ✿ Además, el workflow actualiza el inventario global de historias de usuario `specs/UserStories.md`.
- ✿ **Control (Review Protocol):** *Validation Gate* explícito: el agente debe pedir que la persona desarrolladora valide `specs/features/[feature-slug]/user-stories.md` antes de continuar.

2.3.2. Fase 2: Planificación

En esta fase, las historias se convierten en trabajo implementable, pequeño y trazable.

✿ **/plan-tickets-from-user-stories:**

- ✿ La persona desarrolladora añade la mención al documento de user stories generado en el paso anterior (@).
- ✿ El workflow genera un fichero `specs/features/[feature-slug]/tickets.md` que define los tickets para todas las historias de usuario de ese feature. Para ello aplica **Thin Vertical Slicing**:

para cada historia de usuario, tickets separados para **DB**, **Backend** (API/lógica) y **Frontend**.

- ✿ **Control (Review Protocol):** *Validation Gate* explícito: el agente debe pedir que la persona desarrolladora valide specs/features/[feature-slug]/tickets.md antes de continuar.

✿ **plan-implementation-from-ticket (un plan de implementación por cada ticket):**

- ✿ La persona desarrolladora añade la mención al documento de tickets.md generado en el paso anterior (@).
- ✿ El workflow genera un specs/features/[feature-slug]/plan_[TICKET_ID].md para cada ticket. Este plan incluye un desglose atómico de tareas y pasos a seguir para implementar el ticket.
- ✿ **Control (Review Protocol):** *Validation Gate* explícito: el agente debe pedir que la persona desarrolladora valide cada plan generado dentro de specs/features/[feature-slug]/ antes de continuar.

2.3.3. Fase 3: Implementación

Aquí ocurre la implementación automática, pero **siempre** gobernada por el plan y por un protocolo Git completo que se encuentra descrito en la regla .agent/rules/git-conventions.md.

✿ **/execute-plan:**

- ✿ **Fase 0 (Git Initialization):** si se trata de una implementación nueva, el flujo sincroniza develop y crea la rama de trabajo feat/ para implementación de ticket de feature.
Aviso: Si la implementación viene motivada por el workflow /fix-error, /fix-error habrá creado ya una rama fix/, y este workflow lo detectará esto (idempotencia) y no creará ninguna rama.
- ✿ **Input:** el workflow recibe el plan que debe implementar (plan_[TICKET_ID].md) mediante una mención (@). Además, si el plan implica introducir un dominio nuevo, se usa la skill fastapi-domain-g para crear la estructura por capas (domain, application, infrastructure, presentation) de forma rápida y consistente con arquitectura Clean/Hexagonal (descrita en la regla .agent/rules/a

- ✿ Implementa el ticket siguiendo **estrictamente** plan_[TICKET_ID].md.

- ✿ Aplica ciclo TDD: test → falla → código → pasa → refactor.

- ✿ Inserta los comentarios de trazabilidad obligatorios en el código.

- ✿ Actualiza specs/progress.md (Journaling).

- ✿ **Fase Final (Git Finalization):** antes de terminar:

- ✿ Ejecuta git add.
- ✿ Ejecuta git commit con formato obligatorio (ver regla git-conventions.md).
- ✿ Ejecuta git push al servidor.

Property 2.1

Prohibición operativa: durante execute-plan el agente no puede:

- ✿ ampliar alcance (scope) ni inventar requisitos,
- ✿ cambiar el stack o introducir librerías fuera de techstack.md,

- ❖ tocar archivos no listados en el plan salvo para cumplir tests o integración explícitamente requerida.

2.3.4. Fase 4: Cierre

Por último, el workflow /close-feature es el “punto de control final” antes de dar por terminada una funcionalidad. Su objetivo es asegurar que no queden cabos sueltos técnicos, de calidad o de negocio.

Aquí tienes la lista detallada de todo lo que se ejecuta y comprueba:

1. Verificación Técnica (Tickets)

- ❖ **Escaneo de tickets.md:** se revisan todos los tickets asociados a la funcionalidad (DB, Backend, Frontend).
- ❖ **Estado de finalización:** se comprueba que absolutamente todos los tickets estén marcados como [x] COMPLETED.
- ❖ **Bloqueo:** si falta un solo ticket, el workflow se detiene y pide explicaciones.

2. Puerta de Calidad (Tests)

- ❖ **Ejecución de backend:** corre la suite de pytest (unitarios e integración).
- ❖ **Ejecución de frontend:** corre los tests de Vitest.
- ❖ **Ejecución de E2E:** ejecuta los tests de Playwright para validar el flujo completo del usuario.
- ❖ **Bloqueo:** cualquier test fallido impide el cierre de la funcionalidad.

3. Alineación de Negocio (Objetivos)

- ❖ **Contraste con el diseño:** lee el archivo feature-descr.md.
- ❖ **Validación de objetivos/KPIs:** compara lo implementado con los objetivos de negocio definidos al principio.
- ❖ **Reporte de estado:** clasifica cada objetivo como *Implementado*, *Parcialmente Implementado* o *Diferido*.

4. Sincronización de Documentación (SDD Anti-Drift)

- ❖ **Actualización global:** asegura que specs/PRD.md y specs/UserStories.md reflejen el estado “Shipped” de la funcionalidad.
- ❖ **Actualización de reglas:** verifica que los cambios en la arquitectura o en el esquema de base de datos se hayan reflejado en las reglas correspondientes: @/.agent/rules/ModeloDatos.md y @/.agent/rules/Arquitectura.md.

5. Generación del Reporte de Cierre

- ❖ **Estatus visual:** Cerrado o Parcialmente Completo.
- ❖ **Inventario de entregables:** checklist de Esquema/Migraciones, Endpoints/Lógica y UI/Componentes.
- ❖ **Análisis de deuda:** identifica y lista TODOs pendientes o refactorizaciones menores que se hayan

pospuesto.

6. Journaling (Trazabilidad)

- ❁ **Registro histórico:** añade una entrada obligatoria en `specs/progress.md` con la fecha, el hito (Feature Closed) y los artefactos afectados.

2.4. Intención: arreglar errores (Bugs)

Cuando la intención de la persona desarrolladora es arreglar un error que ha detectado, se sigue un flujo operativo que se compone de un solo paso: invocación del workflow `/fix-error`. El objetivo de este flujo es doble: (1) evitar “ensuciar” el repositorio si el usuario cancela, y (2) formalizar rápidamente los artefactos necesarios para que un workflow de ejecución (p. ej., `/execute-plan`) pueda implementar y validar la solución de forma determinista.

2.4.1. Fase 1: Triage Mental (Zero-Touch)

El agente analiza el error y decide *mentalmente* a qué *Dominio Funcional* pertenece (por ejemplo, User Management). En esta fase:

- ❁ Genera un ID de ticket *virtual* con el formato `<ACRONYM>-BUG-<NN>` (ej., UM-BUG-042).
- ❁ **Nota:** no se toca el disco. Esto evita introducir ficheros o cambios que luego haya que limpiar si el usuario decide no continuar.

2.4.2. Fase 2: Preparación de Entorno (Git Safe-Zone)

Una vez el usuario confirma que se procede, el agente prepara una zona segura de trabajo en Git:

- ❁ Sincroniza el repositorio: `git checkout develop && git pull`.
- ❁ Crea inmediatamente la rama de aislamiento: `git checkout -b fix/um-bug-042`.
- ❁ A partir de aquí, cualquier archivo creado es *seguro* y *desechable* (porque vive en una rama aislada).

2.4.3. Fase 3: Formalización de Artefactos (SDD)

Con el entorno ya aislado, el agente formaliza los artefactos del proceso:

- ❁ **Feature Check:** verifica si existe el fichero `feature-descr.md` correspondiente al feature que está fallando. Si no existe, crea la estructura.
- ❁ **Ticket Creation:** inyecta el ticket dentro del documento `spec/features/[feature-slug]/tickets.md` (dentro de la carpeta de specs correspondiente al feature donde se ha detectado el bug)
- ❁ **Plan Generation:** escribe en el directorio `spec/features/[feature-slug]/` el plan de implementación para solucionar el bug (por ejemplo `plan_UM-BUG-042.md` con la estrategia obligatoria):
 - ❁ Task 1 (RED): crear un test de reproducción que falle.
 - ❁ Task 2 (GREEN): implementar la mínima solución.
 - ❁ Task 3: asegurar la no-regresión.

2.4.4. Fase 4: Delegación de Ejecución

Finalmente, el agente delega la implementación:

- ❖ Invoca automáticamente el workflow hermano: `/execute-plan specs/features/.../plan_UM-BUG-042.md`.
- ❖ Dado que el entorno Git ya está preparado, `execute-plan` detectará que está en la rama correcta y procederá directamente a implementar el código.

👉 Atención

La ventaja principal de que `/fix-error` llame a `/execute-plan` respetar el principio DRY (Don't Repeat Yourself) aplicado a procesos operativos. Si `/fix-error` implementara su propia lógica de ejecución, tendrías dos procesos paralelos (uno para features, otro para bugs) para hacer exactamente lo mismo: escribir código, pasar tests y hacer commits. Al delegar, consigues que un 'hotfix rápido' pase por los mismos controles de calidad que una feature planificada. Además, al evitar duplicidades mejoras la mantenibilidad del proceso SDD al centralizar la lógica de ejecución en un solo workflow y reduces los tokens que se pasan al agente como contexto.

Además de estos dos flujos operativos, GAIA define otros flujos operativos auxiliares pero muy útiles para otros tipos de intenciones comunes en el desarrollo de software.

👉 Atención

Es importante remarcar que GAIA es una aproximación viva, que sigue refinándose continuamente según la experiencia práctica y las necesidades emergentes en el desarrollo de software asistido por IA. Por ello, se espera que nuevos flujos operativos se definan y añadan en el futuro para cubrir otras intenciones comunes que puedan surgir.

2.5. Intención: refactorización de la interfaz de Usuario (UI)

Una intención común es modificar la interfaz de usuario (UI) para mejorar la experiencia del usuario, actualizar el diseño o alinear la apariencia con la identidad de marca. GAIA ofrece workflows específicos para este tipo de cambios, asegurando que se sigan las directrices de diseño y se mantenga la coherencia visual.

Workflows principales: `/refactor-ui-to-brand` o `/refactor-ui-from-reference`

GAIA define dos workflows para refactorizar UI: refactorización a partir de un código/imagen de referencia (`/refactor-ui-from-reference`) o refactorización basada en la identidad de marca de la aplicación (`/refactor-ui-to-brand`).

2.5.1. Refactorización según identidad de marca:

- ❖ **Consulta de marca:** uso obligatorio de la skill `brand-identity`. Prohibido usar colores "hardcoded": deben respetarse tokens de diseño.
- ❖ **Restricción técnica:** seguir `@/techstack-frontend.md`.
- ❖ **Garantía:** se puede obligar a que el agente cree/actualice los tests E2E con Playwright usando el comando `testing-e2e-with-playwright` para asegurar que el cambio visual no rompe funcio-

nalidad.

Property 2.2

Regla de seguridad de UX: un refactor de UI debe ser **funcionalmente neutro** salvo que el ticket especifique cambios de comportamiento. Si cambian flujos, estados o validaciones, eso es un cambio funcional y requiere ticket/plan.

2.6. Intención: cambios en artefactos SDD (PRD, Arquitectura, Reglas)

Cuando la intención es realizar cambios que afectan a los artefactos de alto nivel del proyecto (las reglas, skills o workflows), GAIA establece un flujo operativo específico para garantizar que estos cambios se gestionen con el debido cuidado y revisión. **Workflow principal:** /governance-check + regla/skill/workflow que se quiere modificar (si se sabe), por ejemplo @/review-protocol.md

Protocolo obligatorio:

- ❖ **Impacto:** se activa si el cambio afecta a documentos dentro de specs/ o documentos dentro de .agent/.
- ❖ **Punto de control:** el agente debe parar y presentar un **diff** de lo que pretende cambiar.
- ❖ **Sincronización:** al terminar, se debe actualizar obligatoriamente specs/progress.md para mantener coherente el “estado mental” del proyecto (Journaling).

Property 2.3

Regla de alto riesgo: los cambios en los artefactos SDD del proyecto son *architecture-level decisions*. Por tanto es muy importante asegurar que no se ejecutan en modo automático sin un *Validation Gate* explícito.

Ejercicio 2.1

Clasifica cada cambio y elige el flujo correcto:

1. Un botón nuevo en “Noticias” con un color distinto.
2. Un nuevo feature que cambia el contrato de la API.
3. Un bug intermitente en la pantalla de login.
4. Cambiar la definición del flujo git.
5. Un cambio en el nombre de un endpoint.

Solución

1. UI: brand-identity + /refactor-ui-to-brand y actualizar Playwright.
2. Backend con cambio de contrato: /execute-plan pero activando @/review-protocol.md antes de continuar.
3. Error: /fix-error con test que falle primero y parche mínimo.
4. Gobernanza SDD: /governance-check.
5. Revisión de specs (para ver qué specs está afectando) y de rules (para asegurar que el nuevo nombre no contraviene ninguna regla del proyecto) (/governance-check), seguido de un /fix-

error.

2.7. Intención: technical discovery

Cuando la intención es explorar el código existente para identificar oportunidades de reutilización, extensión o refactorización antes de planificar nuevos tickets, GAIA incorpora un flujo operativo específico llamado `/technical-discovery`. Este flujo es crucial para mantener la salud del código a largo plazo y evitar la duplicación innecesaria.

Posición en el ciclo: se ejecuta **después** de `/plan-user-stories-from-tickets` y **antes** de `/plan-tickets-from-user-stories`.

Problema que intenta paliar: sin discovery, se pasa de Historias de Usuario a Tickets directamente. En proyectos medianos, eso puede provocar que el agente cree piezas nuevas aunque ya existan (componentes UI, modelos, validaciones, endpoints, utilidades, etc.).

Objetivo: obligar al agente a **explorar el código existente** y el modelo de datos para identificar, ANTES de generar los tickets, qué se puede:

- ❖ reutilizar (componentes, hooks, casos de uso),
- ❖ extender (DTOs, tablas, endpoints),
- ❖ o mover a `core/` o `shared/` (refactor proactivo).

Valor: es el antídoto contra el “código espagueti” y la duplicación. Evita crear un `Button.tsx` nuevo si ya hay un primitivo equivalente, o crear una tabla redundante si ya existe un concepto cercano.

Ejemplo 2.1

Si ya existe un sistema de subida de imágenes para Incidencias, y detectamos que la funcionalidad de Noticias es muy similar, el discovery debe identificarlo y proponer reutilización para Noticias, en lugar de generar un flujo paralelo.

2.8. Intención: auditoría de seguridad

Posición en el ciclo: se ejecuta **después** de `/execute-plan` y **antes** de `/close-feature`.

Problema actual: las reglas con las memorias de seguridad (p. ej. `security-backend.md`, `security-frontend.md`) son **pasivas**: existen, pero nada garantiza que siempre se apliquen de forma sistemática tras cada cambio.

Objetivo: un flujo que obligue al agente a asegurar que esas reglas se tienen en cuenta, ejecutando los checklists de seguridad contra el código recién escrito, y que documente el resultado.

Valor: reduce riesgos de forma sistemática:

- ❖ Backend: evita BOLA (Broken Object Level Authorization), validación débil y exposición excesiva de datos.
- ❖ Frontend: previene XSS, almacenamiento inseguro de tokens y secretos en variables de entorno.

Property 2.4

Regla de bloqueo: una feature no puede cerrarse si la auditoría marca riesgos **Críticos** o **Altos** sin mitigación explícita y tests que la respalden.

2.9. Secuencia SDD “Pro” al iniciar algo nuevo

Con estos flujos auxiliares, la secuencia completa de creación de una funcionalidad nueva queda así:

1. /plan-feature-descr-from-user-conversation (qué queremos)
2. /technical-discovery (cómo encaja en lo que ya tenemos)
3. /plan-user-stories-from-features
4. /plan-tickets-from-user-stories (incorporando hallazgos del discovery)
5. /plan-implementation-from-ticket (bucle por ticket)
6. /execute-plan (TDD + Git automático; bucle por ticket)
7. /audit-security-compliance (antes de cerrar)
8. /close-feature (formalizar entrega)

2.10. Conclusión: ecosistema SDD Pro “blindado”

Con estos flujos adicionales, GAIA contribuye a cerrar el círculo de confianza en el desarrollo asistido por IA, asegurando que cada cambio pase por etapas clave de validación y control de calidad:

- ❖ **Discovery** para no reinventar la rueda.
- ❖ **Execution** con TDD y Git automático para producir cambios trazables.
- ❖ **Security Audit** para dormir tranquilo.
- ❖ **Close Feature** para formalizar la entrega.

2.11. Resumen final de GAIA: mapa completo de flujos y artefactos SDD propuestos

GAIA organiza el trabajo en **flujos operativos** que, combinados, forman un sistema de gobierno para que la IA no improvise (requisitos, arquitectura, stack) y para que el proyecto escale sin duplicación ni deriva. El principio rector es: **Planificación en cascada, ejecución ágil**, ampliado con **Discovery, Git gobernado y Auditoría de seguridad** para cerrar el círculo de confianza.

2.11.1 Constitución (prerrequisito permanente)

Artefactos se leen siempre como **fuentes de verdad técnica**:

- ❖ .agent/rules/architecture.md
- ❖ .agent/rules/techstack.md

Artefactos viven en specs/ como **fuentes de verdad de producto y estado (journaling permanente)**:

- ❖ specs/PRD.md

✿ specs/UserStories.md

✿ specs/progress.md

2.11.2 Flujo base para funcionalidad nueva (SDD “end-to-end”)

1. /plan-feature-descr-from-user-conversation

Genera specs/features/[feature-slug]/feature-descr.md y actualiza specs/PRD.md. Activa *Validation Gate* (Review Protocol).

2. /technical-discovery

Explora el código existente y @/ModeloDatos.md para detectar reutilización, evitar redundancia y sugerir refactor a core//shared/.

3. /plan-user-stories-from-features

Genera specs/features/[feature-slug]/user-stories.md y actualiza specs/UserStories.md.

4. /plan-tickets-from-user-stories

Genera specs/features/[feature-slug]/tickets.md aplicando Thin Vertical Slicing (DB-/Backend/Frontend).

5. /plan-implementation-from-ticket (bucle por ticket)

Genera specs/features/[feature-slug]/plan_[TICKET_ID].md. *Validation Gate* humano antes de codificar.

6. /execute-plan (bucle por ticket)

Implementa con TDD + trazabilidad. Actualiza specs/progress.md. Incluye Git Initialization y Git Finalization (add/commit/push).

7. /audit-security-compliance

Audita backend y frontend con checklists de seguridad. Bloquea si riesgo Alto/Crítico. Registra resultado en specs/progress.md.

8. /close-feature

Formaliza la entrega (rama lista para PR, estado coherente y cierre de trazabilidad).

2.11.3 Flujos operativos por tipo de cambio (gobernanza diaria)

✿ Arreglar un error: /fix-error

Test que falla → parche mínimo → commit fix: (ID) descripción → push en rama fix/bug-slug.

✿ Cambios de UI: /refactor-ui-to-brand o /refactor-ui-from-reference + referencia o descripción de brand

Skill brand-identity obligatoria + reglas @/techstack-frontend.md + tests E2E Playwright con testing-e2e-with-playwright.

✿ Cambios artefactos SDD: /governance-check + artefactos afectados

Parada obligatoria + diff propuesto + actualización de specs/progress.md.

👉 Atención

El resultado es un proceso **auditable**: cada línea de código relevante puede rastrearse hasta su ticket y su plan; cada cambio significativo pasa por un gate humano; y cada entrega incorpora una

auditoría de seguridad antes del cierre.

Ejercicio 2.2

Dado el cambio: “añadir funcionalidad de Noticias reutilizando lo que se pueda de Incidencias”, escribe la secuencia de flujos que aplicarías (en orden) e indica en qué puntos ejecutarías el *Validation Gate* humano.

Solución

Una respuesta correcta podría ser:

1. /plan-feature-descr-from-user-conversation (definir alcance y reglas de las noticias).
2. /technical-discovery (encontrar reutilización del sistema de imágenes de incidencias).
3. /plan-user-stories-from-features (historias + Gherkin).
4. /plan-tickets-from-user-stories (tickets DB/Backend/Frontend incorporando reutilización).
5. /plan-implementation-from-ticket (plan por ticket).
6. /execute-plan (TDD + Git automático) por ticket.
7. /audit-security-compliance (verificar XSS, almacenamiento de tokens, autorización).
8. /close-feature.

Validation Gates humanos: tras ejecutar cada paso.

Capítulo 3 GAIA-TDD

3.1. Idea fundamental: La Prueba es la Evidencia

En las aproximaciones SDD en general, y en GAIA en particular, el uso del **Test-Driven Development (TDD)** no es una sugerencia: es un **imperativo sistémico**. La premisa es simple y exigente:

👉 Atención

Regla de Oro (Evidence-Based TDD). No se escribe *ningún* código productivo sin demostrar primero la existencia de un estado fallido (*Red State*) mediante un **test automatizado**.

Esta filosofía conecta con la idea “*Obey the Testing Goat*” popularizada por Harry J. W. Percival. Esta idea significa literalmente **no hacer nada hasta tener un test que falle**. El resultado no es solo “más testing”: es **mejor diseño** y **menos deriva** entre lo que se quiso construir y lo que finalmente se construyó.

a. Cobertura asegurada. En GAIA no existe código “huérfano”: si hay lógica, hay prueba.

b. Diseño intencional. El test obliga a decidir *interfaces, responsabilidades y dependencias* antes de implementar (enfoque *Outside-In*).

3.2. El ciclo de vida: Double Loop TDD

GAIA implementa una variante rigurosa del **Double Loop TDD**: dos ciclos concéntricos que operan a distinta velocidad y nivel de abstracción. La clave didáctica es entender que no compiten: **se complementan**.

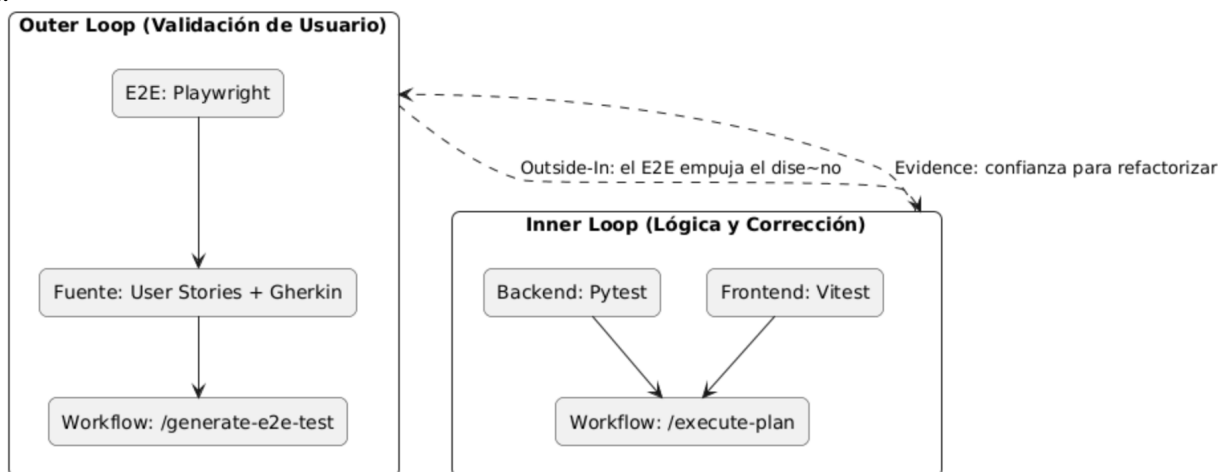


Figura 3.1. Double Loop TDD en GAIA: bucle externo (E2E) y bucle interno (unit/integration).

3.2.1. Bucle externo (Outer Loop): Validación de usuario

- ❖ **Objetivo:** verificar **qué hace** el sistema (comportamiento / historia de usuario).
- ❖ **Herramienta:** Playwright (E2E).
- ❖ **Fuente de verdad:** historias de usuario y criterios de aceptación (Gherkin).
- ❖ **Estado en GAIA:** se gestiona con el workflow `/generate-e2e-test`.

 Atención

Mantenibilidad (POM). Para desacoplar el test de la estructura del DOM, GAIA adopta el *Page Object Model (POM)*: los selectores y acciones de UI se encapsulan en “páginas” reutilizables, reduciendo el coste cuando la UI cambia.

3.2.2. Bucle interno (Inner Loop): Lógica y corrección

- ❁ **Objetivo:** verificar **cómo lo hace** el sistema (implementación y reglas de negocio).
- ❁ **Herramientas:** Pytest (backend), Vitest (frontend).
- ❁ **Estado en GAIA:** se gestiona con el workflow `/execute-plan`.

 Atención

Este bucle debe ser **barato, rápido y determinista**. Si tarda demasiado, el equipo pierde el *flow* y empieza a “adivinar” en lugar de comprobar.

3.3. Implementación operativa de las pruebas en GAIA

GAIA no improvisa: ejecuta una secuencia estricta (Workflows apoyados en skills y reglas) que convierte la intención en evidencia.

3.3.1. Fase A: Diseño y estrategia (Planning)

Workflow: `/plan-implementation-from-tickets`

Antes de codificar, el agente actúa como arquitecto:

- ❁ Identifica qué servicios, componentes o endpoints se deben crear.
- ❁ Decide la **secuencia de pruebas** (qué evidencia se exigirá primero).

3.3.2. Fase B: Ejecución Red/Green/Refactor (Inner Loop)

Workflow: `/execute-plan`

1. RED (falla primero).

- ❁ Crear un test en `tests/unit/` o `tests/integration/` invocando código que aún **no existe**.
- ❁ **Validación:** ejecutar y observar el fallo (por ejemplo `NameError`, `AssertionError`, 404).
- ❁ Si no falla, **se detiene el trabajo**: hay riesgo de falso positivo.

2. GREEN (implementar mínimo).

- ❁ Escribir el código mínimo para pasar el test (YAGNI: nada de sobre-ingeniería).
- ❁ **Validación:** el test pasa a verde.

3. REFACTOR (mejorar sin cambiar comportamiento).

- ❁ Refactorizar con confianza: el test verde protege la intención.
- ❁ Regla práctica: *tres strikes y refactorizar* para eliminar duplicidad.

👉 Atención

Esta disciplina está alineada con la idea de “velocidad alta / velocidad baja” en testing: usar tests rápidos para diseñar y refactorizar, y reservar tests lentos para evidencias críticas.

3.3.3. Fase C: Garantía de calidad E2E (Outer Loop)

Workflow: `/generate-e2e-test`

Cuando la funcionalidad es visible:

- ✿ Traducir escenarios críticos (Happy Path) a Playwright.
- ✿ Priorizar selectores de accesibilidad (`getByRole`, `getByLabel`) frente a CSS frágil.
- ✿ Mantener pocos E2E, pero **muy significativos**.

3.3.4. Fase D: Gestión de errores (Reproduction First)

Workflow: `/fix-error`

Si surge un bug, está prohibido arreglarlo directamente:

1. Escribir un **test de reproducción** que exponga el fallo.
2. Verlo fallar (rojo).
3. Arreglar el bug (verde).
4. Mantener el test como salvaguarda contra regresiones.

3.4. La pirámide de testing en GAIA

El objetivo operativo es proteger el *Holy Flow State*: feedback rápido, estable y accionable.

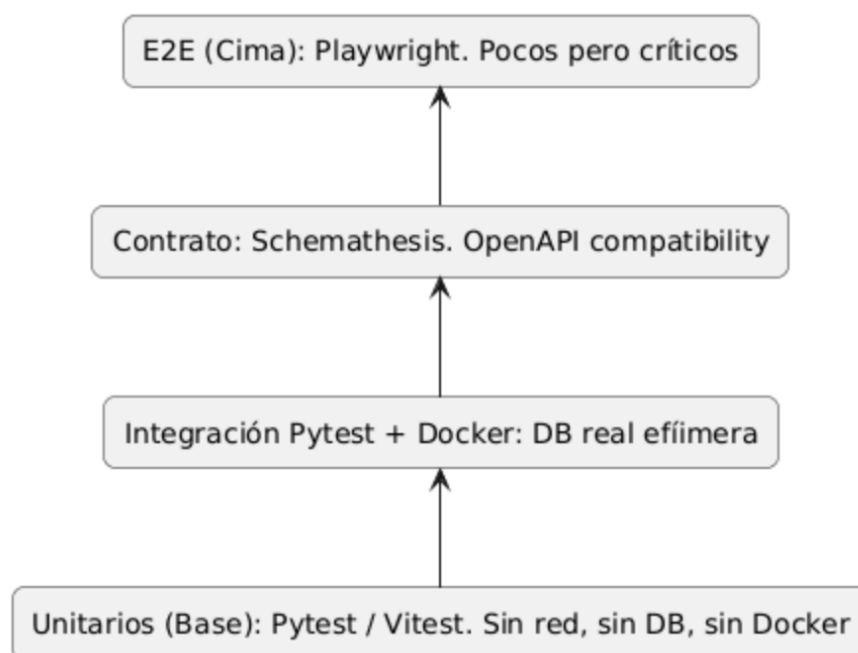


Figura 3.2. Pirámide de testing en GAIA: coste y velocidad determinan qué se prueba y dónde.

- ✿ **Unitarios (base):** aislamiento total; mocks solo en fronteras (I/O, email, APIs externas). Deben correr

en milisegundos.

- ❖ **Integración:** DB real efímera (Docker) y limpieza determinista por test (rollback/truncate).
- ❖ **Contrato:** compatibilidad OpenAPI para detectar *breaking changes* que romperían el frontend.
- ❖ **E2E (cima):** pocos, críticos y robustos; usar `storageState` para evitar logueos repetitivos.

3.5. Glosario de mejores prácticas adoptadas

- ❖ **Mocking:** `unittest.mock` o `vi.mock` solo en límites del sistema. No mockear DB en integración: usar DB dockerizada.
- ❖ **Factories:** preferir `factory_boy` o constructores de datos a fixtures JSON estáticos ("*JSON fixtures considered harmful*") cuando el esquema evoluciona.
- ❖ **Wait for it:** en E2E, nunca `time.sleep`; usar auto-waiting y esperas explícitas de Playwright para evitar *flaky tests*.
- ❖ **Accessibility First:** tests E2E como auditoría básica de accesibilidad con `axe-core`.

3.6. Ejemplo guiado: regla de negocio "no se puede crear un evento sin fecha"

El objetivo del ejemplo no es "hacer FastAPI", sino practicar la mecánica Evidence-Based TDD: **primero evidencia (RED)**, luego **mínimo (GREEN)**, luego **diseño (REFACTOR)**.

3.6.1. Paso 1: RED (test que falla)

Creamos una prueba unitaria que expresa la regla del dominio. Observa que el test invoca una función que *aún no existe*. Eso es intencional.

```
1 # tests/unit/test_events_domain.py
2 import pytest
3
4 def test_cannot_create_event_without_date():
5     with pytest.raises(ValueError) as err:
6         create_event(title="Reunion", date=None)
7
8     assert "date is required" in str(err.value)
```

3.6.2. Paso 2: GREEN (implementación mínima)

Ahora implementamos lo mínimo para pasar el test.

```
1 # app/domain/events.py
2 def create_event(title: str, date):
3     if date is None:
4         raise ValueError("date is required")
5     return {"title": title, "date": date}
```

3.6.3. Paso 3: REFACTOR (mejor diseño, mismo comportamiento)

Si el proyecto está creciendo, este dict empieza a oler. Refactorizamos hacia un modelo explícito, manteniendo la evidencia verde.

```
1 # app/domain/events.py
2 from dataclasses import dataclass
3 from datetime import date as Date
4
5 @dataclass(frozen=True)
6 class Event:
7     title: str
8     date: Date
9
10 def create_event(title: str, date: Date) -> Event:
11     if date is None:
12         raise ValueError("date is required")
13     return Event(title=title, date=date)
```

Atención

Este refactor es un ejemplo de cómo los tests **permiten mejorar el diseño** sin miedo: la especificación ejecutable (el test) protege la intención.

3.7. Actividad práctica

Ejercicio 3.1

Objetivo. Aplicar GAIA-TDD (doble bucle) a una funcionalidad completa, generando evidencia en los dos niveles.

Contexto. Estás construyendo la funcionalidad *Publicar noticia* en una app para una asociación vecinal.

Requisitos.

1. Regla de dominio: no se puede publicar una noticia sin `title`.
2. API: POST /news debe devolver 400 si falta `title`.
3. E2E: un usuario crea una noticia desde la UI y la ve listada.

Entrega (evidencias).

- ✿ Un test unitario que falle primero (RED) y luego pase (GREEN).
- ✿ Un test de integración del endpoint (con DB real o fake controlado).
- ✿ Un escenario E2E en Playwright usando selectores de accesibilidad y POM.

Solución

Solución (estructura sugerida).

1) Unit (RED → GREEN). Expresa la regla como excepción del dominio:

```
1 # tests/unit/test_news_domain.py
```

```

2 import pytest
3
4 def test_cannot_publish_news_without_title():
5     with pytest.raises(ValueError) as err:
6         publish_news(title="", body="texto")
7
8     assert "title is required" in str(err.value)

```

```

1 # app/domain/news.py
2 def publish_news(title: str, body: str):
3     if title is None or title.strip() == "":
4         raise ValueError("title is required")
5     return {"title": title, "body": body}

```

2) Integración (endpoint). El test valida la traducción correcta de la regla a HTTP:

```

1 # tests/integration/test_news_api.py
2 def test_post_news_returns_400_when_title_missing(client):
3     res = client.post("/news", json={"title": "", "body": "texto"})
4     assert res.status_code == 400

```

3) E2E (Outer Loop). Un *Happy Path* con POM y selectores robustos:

```

1 # e2e/pages/news_page.ts
2 export class NewsPage {
3     constructor(private page) {}
4
5     async open() { await this.page.goto("/news"); }
6
7     async create(title: string, body: string) {
8         await this.page.getByRole("button", { name: "New" }).click();
9         await this.page.getByLabel("Title").fill(title);
10        await this.page.getByLabel("Body").fill(body);
11        await this.page.getByRole("button", { name: "Publish" }).click();
12    }
13
14    async shouldSee(title: string) {
15        await this.page.getByRole("heading", { name: title }).isVisible();
16    }
17 }

```

```

1 # e2e/news.spec.ts
2 import { test } from "@playwright/test";
3 import { NewsPage } from "../pages/news_page";
4
5 test("user can publish a news item and see it listed", async ({ page }) => {

```

```
6   const news = new NewsPage(page);  
7   await news.open();  
8   await news.create("Fiesta del barrio", "Este sabado a las 18:00");  
9   await news.shouldSee("Fiesta del barrio");  
10  });
```

Criterio de calidad GAIA. Si en cualquier punto un test “pasa” sin haber visto antes el rojo, deténte: tu evidencia es sospechosa. Si los E2E son frágiles, refactoriza hacia accesibilidad + POM antes de añadir más escenarios. :contentReference[oaicite:3]index=3

3.8. Cierre: por qué GAIA-TDD es más que TDD

GAIA-TDD convierte el testing en **gobernanza técnica**: obliga a producir evidencia antes de producir código, y a estructurar la evidencia según coste/velocidad. El doble bucle evita dos anti-patrones habituales:

- ❁ **Solo unit tests**: rápido pero ciego al comportamiento real del usuario.
- ❁ **Solo E2E**: realista pero lento, frágil y caro de mantener.

El resultado buscado no es “más tests”, sino **menos incertidumbre** y **más confianza** para iterar, refactorizar y entregar con calidad sostenida.